Exno: 7
Date: 13/8/25

Flow Control at Data Link Layer using Sliding Window Protocol.

```
=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===

Enter total number of frames to send : 8
Enter sender's window size : 3
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss): 4

--------------------------------------------

Sender Window : [Frame 0 - Frame 2]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node A --> ACK 2 received. Sliding window forward.

--------------------------------------------

Sender Window : [Frame 3 - Frame 5]
Node A --> Sending Frame 3
Node A --> Sending Frame 4
   (Frame 4 LOST in transit!)
Node A --> Sending Frame 5
Node B --> Frame 4 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 4.

--------------------------------------------

Sender Window : [Frame 3 - Frame 5]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
Node B --> Received Frame 3, sending ACK 3
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node A --> ACK 5 received. Sliding window forward.

--------------------------------------------

Sender Window : [Frame 6 - Frame 7]
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

--------------------------------------------

All 8 frames successfully transmitted from Node A to Node B.
```

* Aim:

To write a program to implement flow control at the Data link layer using the sliding window protocol, and to simulate the flow of frames from one to one another.

* Algorithm:

1. Start the program.
2. Read the total number of frames (N), window Size (w) and lost frames (optional)
3. initialize base = 0 and next seq = 0
4. While base < N:
   * Set the sender window as [base, base+w-1]
   * send frames within the window and increase next seq.
   * If a frame is lost
     ① Receiver send it and later frames.
     ② Sender times out and retransmits from the lost frames.

otherwise:

① Receiver Sender waits for received frames

② Sender slides the window.

5) Repeat until all frames are transmitted and acknowledge.

6) Stop the program.

Program:

```
def simulate(n, w, lost):
    b = ns = 0
    while b < n:
        e = min(b + w - 1, n - 1)
        print("n.. ----- ")
        print(f"Sender Window: [Frames{b} - Frames{e}]")
        while ns < b + w and ns < n:
            print(f"node A --> - Sending Frame{ns}")
            if ns == lost:
                print(f"(Frame{ns} Lost intransit !)")
                ns += 1
        if lost is not None and b <= lost < ns
            print(f"node B --> Frame{lost} missing. Discard.")
            print(f"Node. A --> Time out{lost}.")
```

```
        ns = cost
        lost = None
        continue

for i in range(bins):
    print(f"Node ---> Received Frame {i} send {i}")

    print(f"Node A -> A Ch ans-1 received, sliding")

    b = ns

print("=== Sliding Window Protocal - Flowcontrol\n")

n = int(input("Enter total nos frames to send :"))

w = int(input("Enter senders windows size :"))

while True:

  Try:
    cost = int(input(f"Enter frames number {n-1}.
          Enter -1 for no loss ):"))

    if -1 <= cost <n:
      break

    print(f"please enter a value between -1 and {n-1}")

except ValueError:
    print("Invalid input ! please Enter integer")
```

simulate(n, w, None if lost == -1 else lost).

Result:

The program to implement flow control at the Data link layer using the Sliding Window Protocol was successfully written in python, executed and verified by simulating the flow of frames between two nodes.